

TP N 4 :Synchronisation des processus par des Sémaphores

TP N° 4 :synchronisation des processus par dessémaphores

Sémaphores

les threads peuvent communiquer entre eux par les variables globales. Ils peuvent se synchroniser par attente active, sémaphores, verrous et variables conditionnelles. On s'intéresse dans ce tp par les sémaphores.

Pour utiliser les sémaphores il faut, premièrement, inclure le fichier de déclarations standard :**#include <semaphore.h>**

Deuxièmement, déclarer un descripteur de sémaphore global aux fonctions exécutées par les threads qui l'utiliseront :

```
sem_t sema;
```

Troisièmement, il faut initialiser les descripteurs comme suit :

```
sem_init(&sema,0, 1);
```

Ou le troisième argument correspond à la valeur initiale du compteur du sémaphore (le deuxième argument mis à 0 signifie que le sémaphore peut-être partagé par tous les threads du processus).

Finalement, les opérations P et V sont implémentées par les fonctions sem_wait() respectivement Sem_post(). Donc, une section critique peut être implémentée comme suit :

```
sem_wait(&sema); /* P(sema) */  
/* section critique */  
sem_post(&sema); /* V(sema) */
```

Voila les primitifssémaphores :

```
int sem_init(sem_t *semaphore, int pshared, unsigned int valeur)
```

Création d'un sémaphore et préparation d'une valeur initiale.

```
int sem_wait(sem_t * semaphore) ; Opération P sur un sémaphore.
```

```
int sem_post(sem_t * semaphore) ; Opération V sur un sémaphore.
```

TP N 4 :Synchronisation des processus par des Sémaphores

`int sem_getvalue(sem_t * semaphore, int * sval) ;` Récupérer le compteur d'un sémaphore.

`int sem_destroy(sem_t * semaphore) ;` Destruction d'un sémaphore.

La fonction `sched_yield()`

Un processus peut relâcher volontairement le processeur sans le bloquer en utilisant la fonction «**`sched_yield`**».Le processus est alors placé à la fin de la queue et un nouveau processus est mis en marche.

Exemple :

```
#include <semaphore.h>

int main()
{
    sem_t semaphore;
    int compteur = 0;
    sem_init( &semaphore, 0, 1 ); /* On crée la sémaphore avec la valeur 1 */
    sem_wait( &semaphore ); /* On « prend » la sémaphore */
    compteur++;
    sem_post( &semaphore ); /* On « libère » la sémaphore */
    sem_destroy( &semaphore );
}
```

Le sémaphore binaire Mutex (Mutuel Exclusion)

`int pthread_mutex_init` (`pthread_mutex_t *mutex`, `const pthread_mutexattr_t *attr`);

Initialise le mutex avec les attributs `attr`(NULL pour les options par défaut) et renvoie un code d'erreur.

`int pthread_mutex_destroy`(`pthread_mutex_t *mutex`); Détruit le mutex.

`int pthread_mutex_lock`(`pthread_mutex_t *mutex`); WAIT sur le mutex.

`int pthread_mutex_unlock`(`pthread_mutex_t *mutex`); POST sur le mutex.

Exercice N°1

Vous avez le programme suivant :

```
#include <pthread.h>
#include <stdio.h>
int a =0;
void* increment(void *arg);
void* decrement(void *arg);
int main ( )
```

TP N 4 :Synchronisation des processus par des Sémaphores

```
{
    pthread_tthreadA;
    pthread_tthreadB;
printf("programmeprincipale : a = %d\n", a);
    pthread_create(&threadA, NULL, increment, NULL);
    pthread_create(&threadB, NULL, decrement, NULL);
pthread_join(threadA, NULL);
    pthread_join(threadB, NULL);
printf("programmeprincipale, fin threads : a =%d\n", a);
return 0;
}

void* decrement(void *arg)
{
    a = a - 1 ;
printf("thread B decrement : a=%d\n", a);

return (NULL);
}

void* increment (void *arg)
{
    a = a + 1;
printf("thread A increment : a = %d\n", a);

return (NULL);
}
```

Q1 : Compiler et exécuter le programme. Analyser puis expliquer le résultat d'exécution.

Q2 : C'est quoi la cause de ce problème ? Comment peut-on le résoudre ?

Q3 : Proposer une solution pour résoudre le problème de l'incohérence des données rencontré dans ce programme en utilisant un sémaphore d'exclusion mutuelle pour protéger le variable partagé.

TP N 4 :Synchronisation des processus par des Sémaphores

Exercice N°2

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
sem_t sem1, sem2;

void *T1(void *arg) {
printf("S");
printf("S");
printf("E");
printf("E\n");
pthread_exit(NULL);
}
void *T2(void *arg) {
printf("Y");
printf("T");
printf("M");

pthread_exit(NULL);
}
int main(){
pthread_t tid1, tid2;
pthread_create(&tid1,NULL,T1, NULL);
pthread_create(&tid2,NULL,T2, NULL);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
exit(0);
}
```

Q1 :Initialiser les sémaphores sem1, sem2 dans la fonction main() et insérer dans les fonctions T1 et T2 des instructions sem wait(...), sem post(...) pour que le programme modifié affiche obligatoirement **SYSTEME**.

TP N 4 :Synchronisation des processus par des Sémaphores

Exercice N°3

création de deux threads

On crée trois tâches(threads) pour exécuter chacune des trois fonctions : une affichera des étoiles '*' des dièses'#'et arobase '@'.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>
//Fonctions correspondant au corps d'un thread(tache)
void *etoile(void *inutilise);
void *diese(void *inutilise);

void *arobase(void *inutilise);
```

```
//Remarque:le prototype d'une tâche doit être :
void *(*start_routine)(void *)
```

```
int main(void)
{
pthread_t thrEtoile, thrDiese,thrarobase;
//les ID des de 3 thread
setbuf(stdout, NULL);
//pas de tampon surstdout
printf("Je vaiscréer et lancer 3 threads");
pthread_create(&thrEtoile, NULL, etoile, NULL);
pthread_create(&thrDiese, NULL, diese, NULL);

pthread_create(&thrarobase, NULL, arobase, NULL);

//printf("J'attends la fin des 3 threads\n");
pthread_join(thrEtoile, NULL);
pthread_join(thrDiese, NULL);
pthread_join(thrarobase, NULL);

printf("\nLes 3 threads se sonttermines\n");
printf("Fin du thread principal\n");
pthread_exit(NULL);
return EXIT_SUCCESS;
}

void *etoile( void *inutilise)
{ inti;
char c1 = '*';
for(i=1;i<=200;i++)
{
write(1, &c1, 1);
// écrit un caractère sur stdout(descripteur 1)
}

return NULL;
}
```

TP N 4 :Synchronisation des processus par des Sémaphores

```
void *diese(void *inutilise)
inti;
char c1 = '#';
for(i=1;i<=200;i++)
{
write(1, &c1, 1);
}
return NULL;
}

void *arobase(void *inutilise)
inti;
char c1 = '@';
for(i=1;i<=200;i++)
{
write(1, &c1, 1);
}
return NULL;
}
```

Q1 :compilez ce programme. Quel est le problème rencontré ?

synchronisez les trois threads de ce programme en utilisant les sémaphores pour avoir l’affichage suivant :

-Q2) *#@*#@*#@

-Q3)***###@@@***###@@@

-Q4)**#@**#@**#@**

Exercice N°4

Vous avez le programme de producteur/consommateur suivant :

```
//programme prod_cons.c
#include <semaphore.h>
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>

#define N 3 //taille de tampon

int tampon[N];

void* consommateur(void *);
void* producteur(void *);

int main()
{
pthread_t th1,th2;
```

TP N 4 :Synchronisation des processus par des Sémaphores

```
//créer les threads

//attendre la fin des threads

printf("ici programme principale, fin des threads \n");
return 0;
}

void* producteur(void *depot)
{
    inti=0, nbprod=0, mess=0;
    do {

        //produire message
        tampon[i]=mess;
        printf("\n ici prod. : tampon[%d]= %d\n", i,mess);
        i=(i+1)%N;
        mess++;
        nbprod++;
        sleep(2);

    } while ( nbprod<=5 );
    return NULL;
}

void* consommateur(void *retrait)
{
    int j=0, nbcons = 0, mess;

    do {

        //consommation message
        mess = tampon[j];

        printf("\nicicons.:tampon[%d]=%d\n",j, mess);
        j=(j+1)%N;
        nbcons++;
        sleep(2);
    } while ( nbcons<=5 );
    return (NULL);}
}
```

Q1 :Créer les deux threads pour la fonction producteur et la fonction consommateur.

Q2 : lancer plusieurs fois l'exécution de ce programme. Que remarquez-vous?

Q3 :synchroniser le thread producteur et le thread consommateur selon le schéma suivant en utilisant les sémaphores :

TP N 4 :Synchronisation des processus par des Sémaphores

```
Sémaphore vide = N ;  
Sémaphore pleine = 0 ;  
Sémaphore Mutex1 = 1 (exclusion mutuelle entre producteurs);  
Sémaphore Mutex2 = 1 (exclusion mutuelle entre consommateurs);  
Message Tampon[N];  
Int i, j = 0 ;
```

producteur

```
construire(m)  
P(vide)  
p(mutex1)  
tampon[i]=m  
i=(i+1)mod N  
v(mutex1)  
V(plein)
```

consommateur

```
P(plein)  
p(mutex2)  
m=tampon[j]  
j=(j+1)mod N  
v(mutex2)  
V(vide)  
utiliser(m)
```

Q4 : Compiler et exécuter le programme. Analyser puis expliquer le résultat d'exécution